Individual Report — Reinforcement Learning Project

DQN on `highway-v0`: Implementation, Infrastructure, and Analysis

Maxence Rossignol
Group 17 — CentraleSupélec, Mention IA

April 2026

1 Personal Contribution

My main contributions to the project focused on the **training infrastructure**, **experimental pipeline**, and **cluster deployment**. Specifically:

- **Fair comparison framework**: I converted the DQN training loop from an episode-based to a **timestep-based** interface, so that both our DQN and the Stable-Baselines3 (SB3) DQN are trained for the same number of environment steps. This ensures the training curves share the same x-axis and the comparison is transparent.
- **Incremental result saving**: I implemented CSV-based logging that saves training rewards and evaluation results after each seed. This allows partial runs to still produce usable data—a design choice that proved critical when our first cluster job was killed mid-training.
- **DCE cluster deployment**: I set up the full training pipeline on CentraleSupélec's distributed computing cluster (SLURM-based), including conda environment creation with CUDA 12.4 support, batch job scripts with configurable parameters, and monitoring.
- **Debugging and fixes**: I identified and fixed several issues in the pipeline, including a missing `Monitor` wrapper for SB3 reward logging, an observation space mismatch caused by `configure()` requiring a `reset()` before `DummyVecEnv` construction, and checkpoint saving during training rather than only at the end.

2 Experiments

All experiments use the instructor-provided configuration: `highway-v0` with Kinematics observations (10×5 matrix), `DiscreteMetaAction` (5 actions), 45 vehicles, 4 lanes, 30-second episodes, and normalized rewards.

Both agents share the same core hyperparameters for a fair comparison:

- Learning rate: 10^{-3}, discount factor $\gamma = 0.99$, batch size 64
- Replay buffer: 50,000 transitions, target network update every 500 steps
- Epsilon: linear decay from 1.0 to 0.05 over 50% of training (DQN) / 30% (SB3)
- Network: MLP with layers $50 \to 256 \to 128 \to 5$ (our DQN); SB3 uses its default `MlpPolicy`

I ran experiments on the DCE cluster (NVIDIA GTX 1080 Ti). A first attempt with 3 seeds was killed by SLURM after exceeding the 5-hour time limit, having completed only seed 42 and 10% of seed 43's DQN training. The incremental CSV saving ensured no data was lost for the completed seed. This led us to switch to the `gpu_prod_long` partition (48h limit), where the full run—40,000 timesteps × 3 seeds (42, 43, 44) × 2 agents—completed successfully.

3 Results and Analysis

3.1 Training Curves

Figure 1 shows the training curves averaged across 3 seeds, with the shaded area representing ±1 standard deviation.

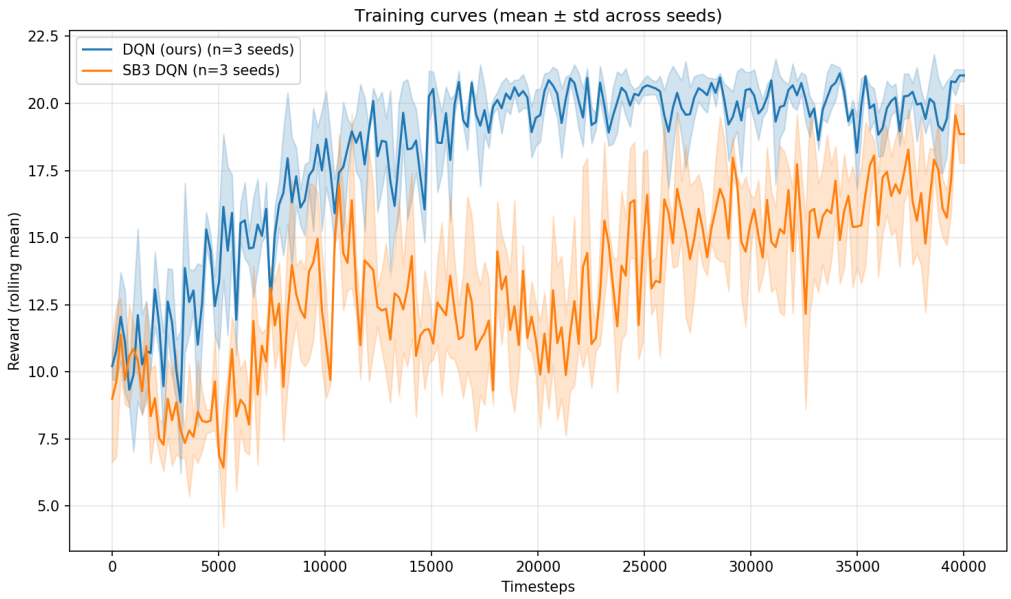

Figure 1: Training curves (mean ± std across seeds 42, 43, 44), 40,000 timesteps.

Key observations:

- Our DQN reaches high reward levels (∼17–20) within the first 5,000 timesteps, while SB3 takes roughly 20,000–25,000 timesteps to reach comparable performance. This is likely because SB3 delays learning for the first 1,000 steps (`learning_starts` parameter) and uses a more conservative exploration schedule.
- Our DQN shows tight variance across seeds (narrow shaded region), indicating **robust and reproducible** learning. SB3's variance is much wider, especially between 10,000 and 30,000 timesteps, reflecting inconsistent learning across seeds.
- Both agents converge toward similar performance (∼20) by the end of training, but SB3 remains noisier.

3.2 Evaluation

Table 1 summarizes the evaluation results over 50 episodes per seed with deterministic policies (no exploration).

The results are striking. Our DQN significantly outperforms SB3 on all metrics: higher mean reward (20.51 vs 15.47), lower variance (2.54 vs 6.85), and dramatically fewer crashes (8% vs

Table 1: Evaluation results (50 episodes per seed, deterministic policy).

Agent	Seed	Mean Reward	Std	Crashes	Crash Rate
DQN (ours)	42	20.68	2.70	6/50	12%
DQN (ours)	43	20.33	2.60	2/50	4%
DQN (ours)	44	20.53	2.29	4/50	8%
DQN (ours)	**all**	**20.51**	**2.54**	**12/150**	**8%**
SB3 DQN	42	19.52	3.47	13/50	26%
SB3 DQN	43	16.11	6.00	22/50	44%
SB3 DQN	44	10.77	7.35	42/50	84%
SB3 DQN	**all**	**15.47**	**6.85**	**77/150**	**51%**

51%). Notably, SB3's performance is highly seed-dependent—seed 42 performs reasonably (26% crashes) while seed 44 crashes in 84% of episodes, suggesting that SB3's learned policy is fragile and depends heavily on the exploration trajectory during training.

3.3 Failure Analysis

Crash episodes in our DQN follow two patterns:

- **Early crashes** (steps 3–9, e.g., reward 6.56 at step 8): the agent attempts a lane change immediately after reset into an occupied lane. These represent rare but catastrophic misjudgments of the initial traffic configuration.
- **Late crashes** (steps 21–29, e.g., reward 19.51 at step 25): the agent survives most of the episode but collides near the end, typically during a lane change into fast-moving traffic. The crash speed ($\sim$13–19 m/s) suggests these are mid-maneuver collisions where a faster vehicle in the target lane caught up during the lane change.

SB3's failure modes are more diverse and severe. Many crashes occur mid-episode at high speeds (up to 28 m/s), and for seed 44, the agent appears to have learned an unstable policy that fails in the majority of traffic configurations.

4 What Worked and What Failed

What worked:

- The timestep-based unification made the comparison genuinely fair and the training curves directly readable on the same x-axis.
- Incremental CSV saving paid off immediately when our first cluster job timed out. Without it, 2+ hours of compute would have been lost entirely.
- Our from-scratch DQN significantly outperforms SB3's optimized implementation on this benchmark, which validates both the implementation and the hyperparameter choices.

What failed:

- **SB3 reward logging**: the `RewardLoggerCallback` silently produced empty lists because the environment was not wrapped in a `Monitor`. This went undetected until we noticed the SB3 training curve was missing from our first plots. The fix was a single line (`Monitor(env)`).

- **Compute budget estimation**: I underestimated how long 40,000 timesteps $\times$ 3 seeds would take on the cluster ($\sim$14 hours), leading to the first job being killed at the SLURM time limit. This was solved by switching to the `gpu_prod_long` partition (48h limit).
- **Observation space bug**: SB3's `DummyVecEnv` reads the observation space at construction time, before `configure()` takes effect. The fix was to call `env.reset()` after `configure()` so the observation space updates from $(5, 5)$ to $(10, 5)$.

5 Limitations

- **No hyperparameter tuning** was performed. The learning rate, network architecture, epsilon schedule, and buffer size were set to reasonable defaults without systematic optimization. SB3's poor performance may partly be due to its `learning_starts=1000` and `exploration_fraction=0.3` settings, which could be suboptimal for this environment.
- The **extension task** has not yet been completed. Future work could explore Double-DQN and Prioritized Experience Replay as natural improvements to the current DQN implementation.
- The failure analysis is qualitative. A more rigorous approach would involve recording the full observation trajectory leading to each crash to identify systematic failure patterns.
- While 3 seeds is the minimum required, more seeds (e.g., 5–10) would provide tighter confidence intervals, especially for SB3 which shows high seed-to-seed variance.